9 - ALGORITMI FONDAMENTALI

Questi algoritmi si chiamano così proprio perché risolvono i problemi **comuni**, come la ricerca di un dato o ordinare vari dati (che sono i più famosi, ma non gli unici).
Sono soluzioni **standard**, riconosciute ormai come **corrette**.
Come sappiamo un buon programmatore deve scegliere la soluzione **ottimale** tra le varie soluzioni che esistono per risolvere un problema. Per scegliere la soluzione migliore ci si basa sulla **complessità computazionale** degli algoritmi; tipicamente gli algoritmi con *CC* più bassa (più **efficienti**) hanno una complessità implementativa più **alta**.

COMPLESSITA' DI UN ALGORITMO

La complessità di un algoritmo è la misura di quanto è **complesso** per l'elaboratore eseguire il determinato algoritmo.
Questo si basa su due tipi di**risorse** utilizzate dall'algoritmo:

- **Spazio**: quantità di memoria occupata durante l'esecuzione
- **Tempo**: quantità di tempo impiegata per ottenere una soluzione
  Minori saranno le risorse usate da un algoritmo, **minore** sarà la sua complessità computazionale.
  Calcolare il tempo per un programmatore prima di testarlo non potrà mai essere un numero certo, per questo per calcolare il **tempo** necessario all'esecuzione di un algoritmo, per convenzione, si calcola il numero di volte che viene ripetuta l'**operazione principale** (lo scambio, la somma, ecc...).

```
for(int i=0; i<n; i++) {
        // si dice che questo codice ha complessità «n»
        // perché il frammento viene eseguito n volte
}
```

  Tipicamente quando si calcola la complessità si cercano le istruzioni *dominanti*, cioè quelle che vengono eseguite più volte. Spesso (non sempre) la complessità degli algoritmi è data dal **numero di operazioni** che vengono effettuate nei cicli.

LIVELLI DI COMPLESSITA' DI UN ALGORITMO

I livelli di complessità sono crescenti:

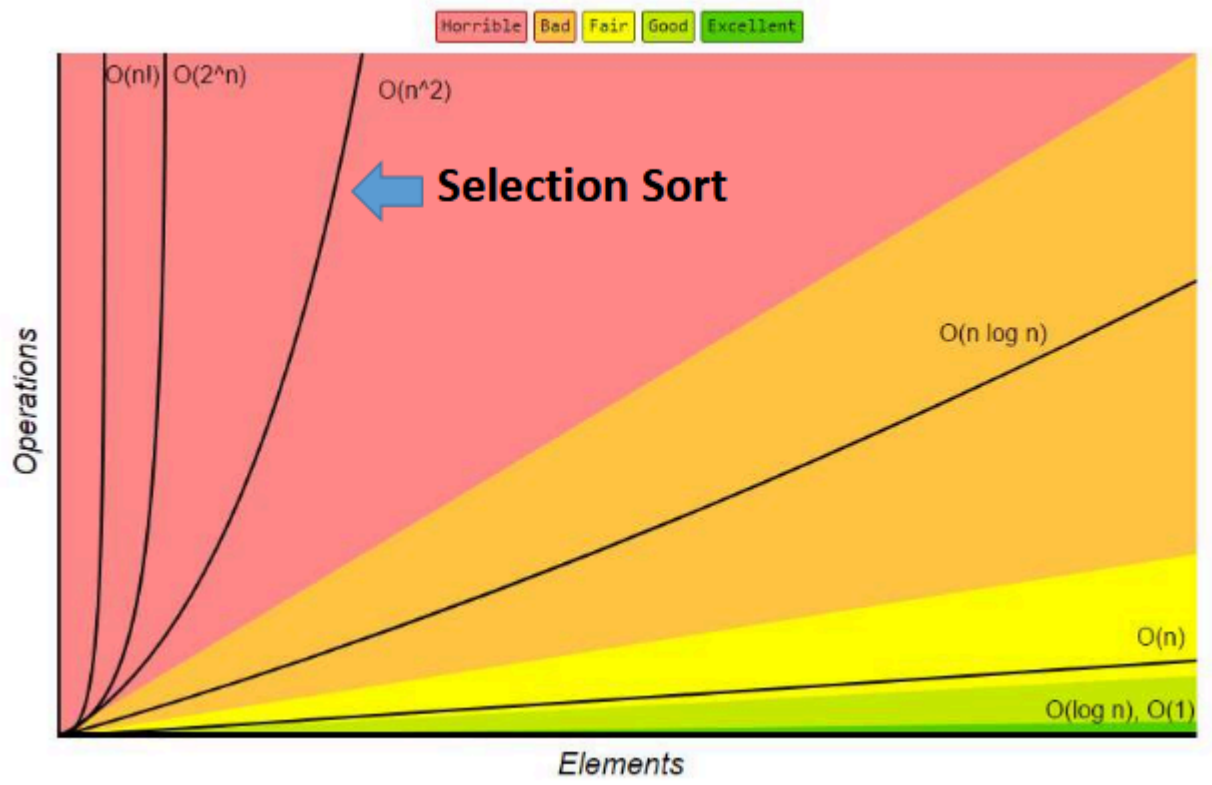
- Costante $O(1)$
- Logaritmica $O(\log n)$
- Lineare $O(n)$
- nLog
- Polinomiale $O(n^3)$

- Esponenziale $k > 1$
  Gli algoritmi più **comuni** hanno una complessità **polinomiale e/o lineare** ed hanno una complessità ottimale per essere risolti. Gli algoritmi più complessi e **non trattabili** hanno una complessità **esponenziale**, mentre gli algoritmi più **efficienti** hanno una complessità **logaritmica**.

Quando si parla di complessità computazionale bisogna dividere **diversi casi**:

- **MIGLIORE**: corrisponde alla configurazione iniziale che comporta solo il **minimo** numero di esecuzioni dell'operazione principale. (come un array già ordinato)
- **PEGGIORE**: corrisponde alla configurazione iniziale che comporta il **massimo** numero di esecuzione dell'operazione principale.
- **MEDIO**: se il valore da ricercare ad esempio si trova al centro del vettore

GRAFICO DEI LIVELLI DI COMPLESSITA':



RICERCA

Il problema che si riscontra quando necessitiamo di un algoritmo di ricerca è il voler determinare se un certo elemento *x* compare in un certo insieme di *n* dati.
I possibili esiti da un algoritmo di ricerca sono o un **elemento non presente** o un **elemento trovato**, in cui si restituisce la posizione.
Normalmente una funzione di ricerca deve seguire questi **parametri**:

```
int ricerca(int valore, int vettore[], int n)
```

RICERCA LINEARE ESAUSTIVA

Questo metodo è utilizzabile quando si può accedere in sequenza a tutti gli elementi nella lista. Non viene posta nessuna ipotesi di **ordinamento**.
La **complessità** di questo algoritmo è basata sul numero di confronti (cioè sul numero di **cicli effettuati**).

- In questo caso il caso migliore è $O(n)$ perché effettua tutti i cicli.
- Il caso peggiore è $O(n)$ poiché si devono controllare comunque tutti gli elementi fino alla fine.
- Il caso medio è $(n+1)/2 \rightarrow O(n)$ supponendo una distribuzione casuale di valori, si ritornerà sempre allo stesso risultato precedente.
  Per migliorare questo algoritmo si introduce il contesto di **sentinella**, ovvero fermare la ricerca una volta che l'elemento è stato individuato, restituendo la prima **posizione di occorrenza**.
  Questa miglioria è utile solo se il valore è **unico**.

```
j ← 0
posizione ← -1
mentre (j < n) e (posizione < 0)
  se lista(j) = x allora posizione ← j
  j ← j + 1
```

Esempio Sentinella:

```
int ricerca(int a[], int n, int x){
    j=0;
    posizione==-1;
    while((j<n) && (posizione < 0)){
        if(a[j]==x) posizione=j;
        j=j+1;
    }
    return posizione;
}
```

La complessità in questo modo varia, basandoci sempre sul numero dei confronti, il caso **migliore** diviene $O(1)$, ovvero l'elemento si trova in prima posizione. Il caso **peggiore** è $O(n)$, ovvero l'elemento è in ultima posizione. Il caso medio rimane invariato.

RICERCA BINARIA (DICOTOMICA)

Questo algoritmo fondamentale è più efficiente e con *CC* più bassa. Il suo unico **vincolo** è che applicabile solo ad insieme di dati **ordinati**, quindi per guadagnare efficienza nella ricerca, si richiede a monte un'applicazione di un algoritmo di ordinamento.
Legge della ricerca dicotomica:
Confrontare il valore cercato con quello al centro della lista, e se non è quello cercato, basarsi sul confronto per escludere la parte superflua e concentrarsi sull'altra parte.

Se l'elemento centrale è quello cercato allora l'elemento cercato è in quella posizione, altrimenti se è minore di quello cercato bisogna analizzare la metà successiva, altrimenti bisognerà analizzare la metà lista precedente, se continuando a dividere si arriva ad un elemento singolo che non corrisponde allora, solo in questa condizione, capiremo che l'elemento non c'è, per questo si cicla finché la parte di lista da analizzare contiene più di un elemento e quello da cercare non è stato ancora trovato.

Codice ricerca dicotomica:

```
int ricerca(int a[], int n, int x){
    posizione=-1;
    first = 0; last = n-1;
    while((first <= last) && (posizione=-1)){
        j=(first+last)/2; //arrotondato per difetto
        if(lista[j]==x) posizione=j; //trovato
        else{
            if(x>lista[j]) //se il target è maggiore del mediano
                first=j+1;
            else
                last=j-1;
        }
    }
    return posizione;
}
```

La complessità della ricerca dicotomica può variare dal:

- numero minimo di cicli che è pari ad 1, ovvero la variabile è in mezzo alla lista
- numero massimo di cicli che è pari $(\log_2 n) + 1 = O(\log_2 n)$. Per esempio se avessimo un $n = 128$:
  - nel **primo** ciclo avremo 128 elementi da esaminare, al termine gli elementi da esaminare si dimezzeranno
  - nel **secondo** ciclo avremo 64 elementi da esaminare e così via
  - ...
  - al **settimo** ciclo avremo 2 elementi, per uscire dobbiamo rimanere con un elemento che sarà quello da cercare o comunque l'unico elemento con cui si potrà uscire dal ciclo.
  - all'**ottavo** ciclo troveremo l'elemento e sarà il suo ciclo di termine massimo. Con un totale di complessità pari a $(\log_2 128) + 1 = 8$
  Si evince quindi che il tempo richiesto per trovare un elemento aumenta in modo logaritmico rispetto alla dimensione della lista.

ALGORITMI DI ORDINAMENTO

L'obiettivo è disporre gli elementi in una precisa **relazione d'ordine**.
La *CC* si basa sul tipo di dato che abbiamo dichiarato, in base a ciò l'ordinamento può essere **numerico** e **alfanumerico**, e possono essere entrambi **crescenti/decrescenti**.

Anche in questo caso non esiste un **algoritmo migliore in assoluto** ma dipende dal contesto del problema in cui ci ritroviamo.
L'attività di ordinamento occupa in media il 30% del tempo del calcolo dell'elaboratore, per questo è un attività di elaborazione **importante**.
Gli algoritmi di ordinamento si dividono in due operazioni differenti tra gli elementi, **confronti** e **scambi**.
L'ordinamento si dive anche in:

- **Esterni**: usando un array di appoggio, in cui si farà doppia occupazione di memoria e la necessità di copiare il risultato nell'array originale.
- **Interni**: l'ordinamento viene eseguito sullo stesso array da ordinare.
  I più famosi e utilizzati sono:
- **Per selezione (Selection Sort)**
- **A bolle (Bubble Sort)**
- **Per inserzione (Insert Sort)**

SELECTION SORT

Esso è basato sul concetto di **minimi successivi**, ovvero:

1. trovare il **più piccolo elemento** dell'insieme e porlo in prima posizione
2. trovare il **più piccolo dei rimanenti (n-1)** elementi e sistemarlo in seconda posizione
3. ripetere finché si trovi e collochi il **penultimo elemento**
4. l'ultimo elemento sarà automaticamente sistemato

```
void selectionSort(int a[], int n){
    for(i=0; i<n-1; i++){
        min=a[i];
        p=i; //p=posizione del minimo, min=val.minimo
        for(j=i+1;j<n;j++){ //trovare il minimo
            if(a[j]<min){
                min=a[j];
                p=j;
            }
        }
        a[p]=a[i]; //una volta individuato il minimo
        a[i]=min; //effetto lo scambio tra i valori
    }
}
```

	Inizio	I	II	III	IV	V
array(1)	44	44	11	11	11	11
array(2)	33	33	33	22	22	22
array(3)	66	66	66	66	33	33
array(4)	11	11	44	44	44	44
array(5)	55	55	55	55	55	55
array(6)	22	22	22	33	66	66

Complessità del Selection Sort

La complessità totale è data dalla complessità dei due cicli.
*Trattando i **confronti***:
Il **ciclo esterno** si ripete n-1 volte, mentre il **ciclo interno** ricerca il minimo nella parte dell'array non ancora ordinata. La complessità è **quadratica** $O(n^2)$
I confronti tra gli elementi nell'array si misurano tramite la formula $\frac{n(n-1)}{2}$
*Trattando gli **scambi***:
Lo scambio viene effettuato solo quando viene trovato il **minimo**, uno per ogni passo di ordinamento del sotto-array.

Ogni ciclo scorre tutta la parte **non ordinata**, non trae quindi vantaggio da un eventuale **pre-ordinamento**. Il suo vantaggio è che vengono effettuati pochi scambi, ogni scambio di complessità $O(1)$ poiché richiedono solo 3 passaggi.

BUBBLE SORT

Rispetto all'algoritmo precedente, questo algoritmo è **espressamente** basato sugli scambi degli elementi. Gli elementi più piccoli salgono verso l'alto come bolle, ad ogni passo si **ordina** un elemento. Il numero di scambi quindi sarà nettamente maggiore rispetto al selection sort.

	Inizio	I/1	I/2	I/3	I/4	II
array(1)	4	4	4	4	4	1
array(2)	3	3	3	3	1	4
array(3)	6	6	6	1	3	3
array(4)	1	1	1	6	6	6
array(5)	5	2	2	2	2	2
array(6)	2	5	5	5	5	5

Complessità del Bubble Sort

I vari casi di complessità del bubble sort varia sul:

- **caso migliore**: eseguendo un singolo passo di ciclo quando la **lista è già ordinata**, con complessità $O(n)$
- **caso peggiore**: è l'esatto opposto, ovvero quando nessun elemento è posto in ordine, avendo un numero di passi pari a $n-1$. Vengono eseguiti un numero di confronti in maniera **decrescente**, perché ad ogni passo una parte sarà sempre ordinata, gli *n-i* scambi sono in tutto $(n-1)*n/2->O(n^2)$, notando un incremento di scambi altamente maggiore della Selezione.
- **caso medio**: ove gli scambi siano pari alla metà dei confronti, $O(n^2)$
  Il bubble sort è chiaramente inferiori agli altri metodi, nel caso peggiore il numero di confronti sarà pari all'ordinamento per selezione, ma con scambi maggiori. E' molto veloce per gli insiemi con alto grado di preordinamento. Per delineare un confronto tra i metodi si può affermare che:
- **Se l'insieme è pre-ordinato**: $BubbleSort > SelectionSort$
- **Se l'insieme non è pre-ordinato**: $BubbleSort < SelectionSort$

INSERCTION SORT

Si basa sul metodo eseguito nei giochi di carte per mischiare il mazzo.
Questo algoritmo **ricerca** la giusta posizione d'ordine di ogni elemento rispetto alla parte già ordinata. Gli elementi da ordinare vengono considerati uno per volta, si confronta l'elemento *n* con tutti quelli della parte ordinata e lo si colloca nella giusta posizione, facendo scalare gli altri o di uno a destra o di uno a sinistra. Al primo passo avremo due elementi ordinati, quindi dopo *n-1* passi avremo tutti gli elementi ordinati.
In totale questo algoritmo effettua **n-1 passi**.

	Inizio	I	II	III	IV	V
array(1)	40	30	30	10	10	10
array(2)	30	40	40	30	30	20
array(3)	60	60	60	40	40	30
array(4)	10	10	10	60	50	40
array(5)	50	50	50	50	60	50
array(6)	20	20	20	20	20	60

Ad ogni passo dell'algoritmo una parte sarà chiamata **parte del vettore ordinata** e l'altra zona è la **parte non ordinata**.
La scansione della posizione in cui inserire il nuovo elemento nella parte ordinata è **sequenziale**. Al ciclo *i*, avremo *i+1* **elementi ordinati** e *n-(i+1)* **elementi non ordinati**.
Anche questo algoritmo fa uso di una **sentinella**, per comprendere se l'elemento è stato inserito o meno al posto giusto, se non è stato inserito continua fin quando non lo trova, e se è minore del precedente allora si scalerà il precedente, altrimenti viceversa, e si posizione

poi fin quando non trova la posizione giusta. Bisogna anche controllare che se questo elemento analizzato arriva in prima posizione, non dovrà mai essere più spostato da lì, rendendo successivamente **true** la sentinella.

Inserction Sort in C:

```
void inserction_sort(int x[], int n){
    int i,j,app;
    for(i=1;i<n;i++){ //ciclo che scorre tutti gli elementi partendo dal
secondo
        app=x[i]; //salva il valore da posizionare
        j=i-1; //memorizza l'indice dell'ultimo valore ordinato
        while(j>0 && x[j]>app){
            x[j+1]=x[j];
            j--; //se l'elemento è più piccolo di quello da posizionar
        } //ciclo per individuare la posizone corretta
        x[j+1]=app; /* esce dal ciclo quando ha trovato la posizione
    }                * per l'elemento, dunque inserisce il valore lì*/
    return;
}
```

Complessità Inserction Sort

I passi saranno sempre pari a $n-1$, con un numero di scambi che è pari ad ogni confronto effettuato, salvo l'ultimo.

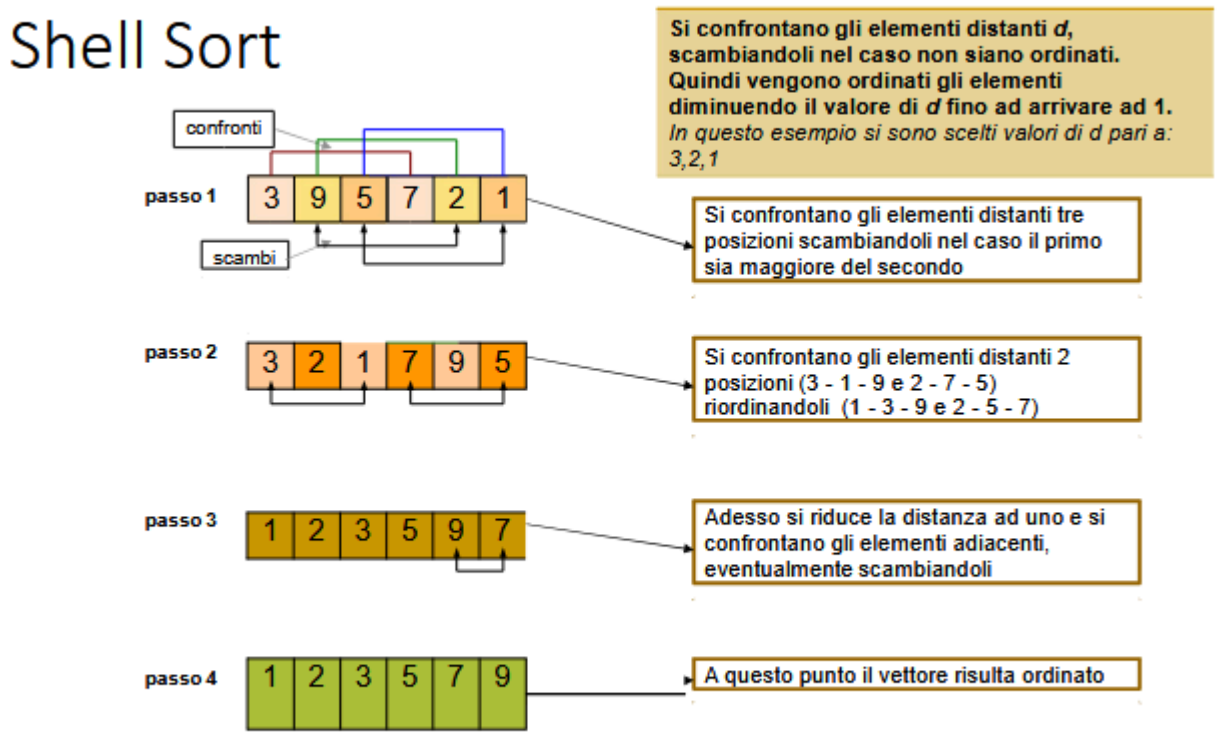
- **Caso ottimo**: la lista è già ordinata, avendo $n-1$ confronti e 0 scambi, avendo lo stesso risultato del metodo a bolle nel caso ottimale.
- **Caso pessimo**: la lista non è per nulla ordinata, con un totale di confronti e scambi pari a $i-1$, $\frac{(n-1)*n}{2}$.
- Anche nel **caso medio** la complessità è uguale a quella del caso pessimo, la quale complessità di entrambi è pari a $O(n^2)$.
  Questo algoritmo è **efficace** per piccole sequenze e/o sequenze **parzialmente ordinate**, riducendo così il numero degli scambi, diminuendo anche il tempo di esecuzione, dato il grande peso degli scambi, rispetto al confronto.
  Ad ogni passo la porzione ordinata **cresce di una unità**, differentemente la porzione disordinata **decresce** di una unità.

TECNICHE AVANZATE DI ORDINAMENTO

Gli algoritmi di ordinamento di base, come abbiamo notato, hanno un livello di complessità elevata per risolvere problemi banali della realtà odierna.
E' necessario introdurre degli algoritmi che abbiano una complessità **lineare**.

SHELL SORT

Questo è un **algoritmo evoluto**, ed è basato sul concetto di **riduzione degli incrementi**. Si confrontano tutti gli elementi che si trovano ad una **distanza *d*** e si continua **riducendo** il valore *d* fino ad arrivare ad elementi adiacenti *d=1*.
Questo algoritmo va a modificare il basico bubble sort, poiché in quest'ultimo si confrontano solo gli elementi adiacenti.



L'ultimo passo sarà identico ad un bubble sort, ma si avrà creato a monte un array pre-ordinato manualmente.

Come si sceglie la distanza?

Valutare il valore di *d* è molto complesso, bisogna tenere a mente che l'ultimo passo deve avere *d* **sempre pari a 1**.
Le sequenze tipicamente utilizzate sono: 9,5,3,2,1
Per **dogma** si preferisce non usare distanze pari alle potenze di 2.
E' un algoritmo efficiente, poiché **diminuisce** gli scambi da effettuare.

Implementazione Shell Sort

```
void ShellSort(int* vett, int dim){
    int i,j,gap,k;
    int x,a[5]={9,5,3,2,1};
    for(k=0;k<5;k++){
        gap=a[k];
        for(i=gap;i<dim;i++){
```

```
            x=vett[i];
            for(j=i-gap;(x<vett[j]) && (j>=0); j=j-gap){
                vett[j+gap]=vett[j];
                vett[j]=x; //scambio elementi
            }
        }
    }
}
```

Possiamo effettuare più scambi a ciclo, nel primo ciclo si confronta *x* con *vett[0]*, quindi primo elemento con l'elemento pari al gap, ovvero in questo caso di defautl sarà *vett[9]*. Nel ciclo successivo i valori si incrementano, quindi *vett[1]* con *vet[10]* (se esiste), ecc...

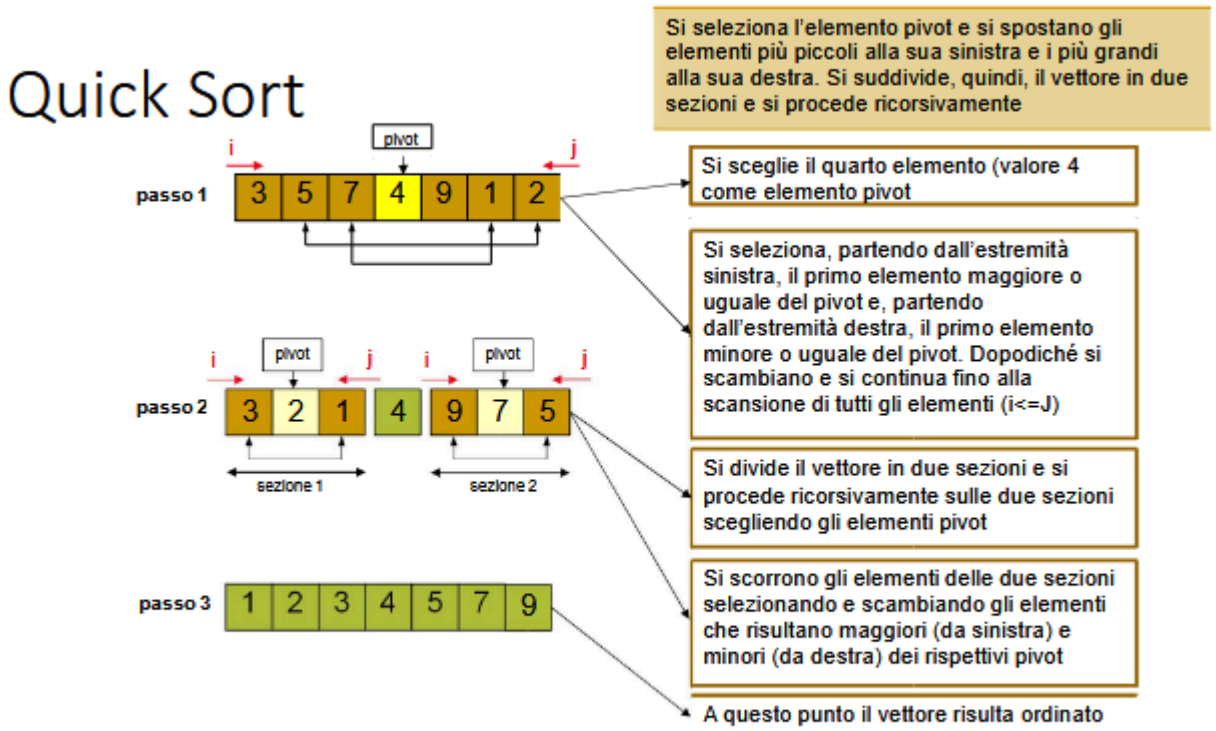
Complessità del shell sort

La complessità media è pari a $O(n \log_2 n)$, ma ciò dipende dalla distribuzione dei dati. La complessità rimanere di questo livello anche nel caso peggiore, quindi tende ad ottenere prestazioni migliori.
Intuitivamente si comprende che gli elementi vengono spostati più **rapidamente**, utilizzando **meno confronti**.

QUICK SORT

E' un algoritmo che si basa sulla **ricorsione**, è un algoritmo che richiama sé stesso, è più semplice ed elegante.
La sua *CC* è pari a $O(n \log n)$ nel caso ottimo e nel caso medio. Per quanto riguarda il caso peggiore, non si ha una miglioria rispetto agli algoritmi precedenti, mantenendo una complessità polinomiale pari a $O(n^2)$.

Questo algoritmo è definito come un **concetto di partizione**, la procedura generale consiste nella selezione di un valore del vettore analizzato, questo valore viene definito **pivot** e suddividerà il vettore in **due sezioni**, la prima formata da tutti i valori inferiori al pivot e nella seconda sono presenti tutti quelli maggiori.
Questo processo viene ripetuto per ognuna dei settori rimanenti fino all'ordinamento completo.



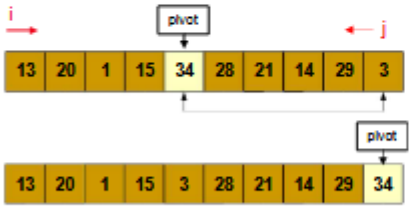
In questo caso ci basiamo sul caso migliore in cui è possibile prendere il pivot (ovvero perfettamente a metà), ma bisogna scegliere con parsimonia se prendere il pivot in base alla posizione (in un array lungo) o in base al valore (in un array breve con valori distinti e non uguali); la scelta del pivot **influisce totalmente** il comportamento del QuickSort.

Analisi del Pivot

La selezione del pivot ha un impatto determinante sulle prestazioni del Quick Sort. Il caso peggiore si verifica quando il pivot scelto è l'elemento minimo o massimo, portando a una suddivisione altamente sbilanciata dell'array.
Il caso peggiore nella scelta del pivot è la creazione di una scissione formata dalla medesima dimensione del vettore iniziale - 1, mentre l'altro lato ha una dimensione unitaria.
Bisogna avere molta **parsimonia** e a volte anche fortuna e buone conoscenze progettuali per determinare la scelta del pivot. Solo nel caso in cui conoscessimo a priori il vettore possiamo risalire all'**elemento mediano**, ovvero la scelta migliore.

Quick Sort – Caso peggiore

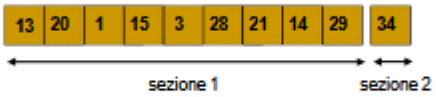
Se scegliamo come pivot l'elemento di posto centrale → indice m=(inf+sup)/2)
Il pivot casualmente coincide con l'elemento massimo del vettore.



L'indice i viene incrementato **fino a quando non viene trovato un elemento più grande o uguale al pivot**. Nel nostro esempio l'indice i si arresta in corrispondenza del pivot.

L'esempio mette in evidenza il motivo di incrementare l'indice i fino a quando si trova un elemento più grande o uguale al pivot. Se non ci fosse la condizione uguale, nel nostro esempio l'indice i verrebbe continuamente incrementato oltre la dimensione del vettore.

Per quanto riguarda l'indice j esso non viene spostato in quanto l'elemento j-esimo è inferiore al pivot.



Siccome **i > j**, la prima passata è finita: otteniamo due sezioni di cui una ha dimensione 1

Il metodo migliore per scegliere il pivot è la **scelta casuale**, selezionando l'elemento che occupa la posizione centrale.
Da questo si evince che non sempre questo algoritmo può essere effettivamente una miglioria, bisogna conoscere e studiare il problema che si sta analizzando per determinare la giusta scelta degli algoritmi da implementare.

QuickSort nel C:

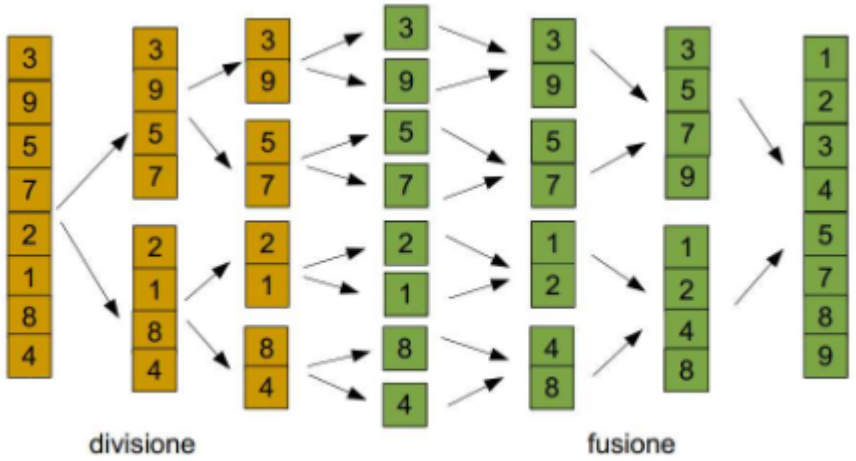
```
void quickSort(int v[],int l, int r){
    int i;
    if(r<=1) return;
    i=partition(v,l,r);
    quickSort(v,l,i-1);
    quickSort(v,i+1,r);
}

int partition(int v[],int l, int r){
    int x,i,j,temp;
    int p=(l+r)/2;
    x=v[p];
    i=l-1;
    j=r+1;
    while(i<j){
        while(v[--j]>x){
        while(v[++i]<x){
            if(i<j){
                temp=v[i];
                v[i]=v[j];
```

```
                v[j]=temp;
            }
        }
        return j;
    }
}
```

Questa funzione restituisce l'indice *j*, il quale rappresenta il punto in cui termina la metà a sinistra dell'array partizionato, tutti gli elementi da *v[0] a v[j]* sono minori o uguali a pivot.

MERGE SORT

Il merge sort è un algoritmo evoluto di fusione con *CC* pari a $O(n\log(n))$ in tutti i suoi casi. Anche questo come il precedente è un algoritmo ricorsivo, basato sul principio del **divide et impera**, sfruttando il concetto di *merging* degli array ordinati.
Il merge utilizza uno **spazio ausiliario proporzionale a N**, inoltre le risorse di tempo e spazio impiegate **non dipendono dall'ordinamento iniziale** del file di input. Il merge sort è particolarmente indicato per i dati di grandi dimensioni già parzialmente ordinati, poiché la sua complessità rimane invariata indipendentemente dalla distribuzione dei dati.



Merge sort nel C:

```
void mergeSort(int a[],int l, int r){
    if(r<=l) return;

    int m=(r+l)/2;
    mergeSort(a,l,m);
    mergeSort(a,m+1,r);

    merge(a,l,m,r);
}

void merge(int a[],int l, int m, int r){
    int i,j,k,*aux;
    aux=(int*)malloc((r-l+1)*sizeof(int));
```

```
    for(i=m+1;i>l;i--){
        aux[i-1]=a[i-1];
            for(j=m;j<r;j++){
                aux[r+m-j]=a[j+1];
                for(k=l;k<=r;k++){
                    if(aux[j]<aux[i]) a[k]=aux[j--];
                    else a[k]=aux[i++];
                }
            }
    }
}
```

Questa implementazione del codice fa uso di un array ausiliario di dimensione proporzionale all'output, per fare ciò il secondo array viene trascritto in maniera inversa alla fine del primo. Nella **prima funzione** si prende un array in input e i due corrispettivi indici dx e sx. La funzione richiama se stessa per poter suddividere in due l'array in base ai casi.

- caso base: se l'indice dx e sx sono uguali o sx è minore di dx, significa che il singolo array è vuoto o contiene un singolo elemento. In questo caso è già ordinato e ritorna immediatamente.
- calcolo del punto medio: il punto medio viene calcolato con $l + r/2$. Questa operazione ci permette di suddividere l'array nelle due metà, da *l* a *m* il primo array e il secondo da *m+1* a *r*.
- La funzione richiama se stessa successivamente per ordinare ricorsivamente le due metà del sotto array.
- Tramite l'uso della funzione merge si andranno ad unire i due nuovi array.
  La **seconda funzione** crea un array di ausilio della stessa dimensione del sotto-array. Questo array seve come spazio di lavoro per memorizzare temporaneamente gli elementi durante la fusione.
  Successivamente si andranno a copiare gli elementi da sinistra a destra nel primo ciclo *for*, andando da *m+1* a *l*, copiando nel vettore ausiliare in ordine inverso questi elementi man mano che ci si avvicina a destra. Corrispettivamente nel secondo ciclo *for* andremo a copiare gli elementi da destra a sinistra, sempre in ordine inverso. Infine si ritorna in un ciclo *for* principale che scorre tutta la lunghezza dell'array iniziale, riempiendo l'array originale con i valori ordinati. Durante questo ciclo vengono confrontati i valori dalla parte sinistra e dalla parte destra del vettore ausiliare.